Contrato entre nodos — Proyecto Grupal 1

CE1110 Análisis de Señales Mixtas — Sistema distribuido de reducción adaptativa de ruido en tiempo real

Referencia técnica común para cualquier tarea del proyecto. Todo nodo, script de prueba o simulación debe respetar este documento. Si algo necesita cambiar, se cambia aquí primero y se avisa al grupo.

Estado: borrador para aprobación del grupo.

- Lo marcado como **[Tarea 1]** viene del informe de la Tarea 1 (sección 6.7) y ya estaba definido.
- Lo marcado como **[Propuesta]** es nuevo y debe aprobarse antes de programarlo.

1. Parámetros del sistema [Tarea 1]

Parámetro	Valor
Frecuencia de muestreo fs	8000 Hz
Tamaño de FFT N	1024 muestras (Δf = 7.8125 Hz)
Avance (hop)	512 muestras (50 % de solape), un bloque cada 64 ms
Ventana espectral	raíz de Hann periódica, en análisis y en síntesis
Formato numérico interno	float32, señal normalizada en [−1, 1)
Retardo del FIR pasa bajas	18 muestras (2.25 ms)
Retardo de reproducción objetivo	256 ms (cuatro avances)
Entorno de firmware	PlatformIO, `framework = espidf`, C++, biblioteca ESP-DSP

Conversión ADC y DAC [Propuesta]

- **ADC (nodo 1, 12 bits):** `x = (código − 2048) / 2048`, después de quitar el nivel DC estimado.
- **DAC (nodos 2 y 3, 8 bits):** `código = redondear(127.5 + 127.5·y)`, limitado a 0–255. Cada vez que se limita se suma 1 al contador de saturaciones.

2. Pines de cada placa [Propuesta]

Función	Nodo 1	Nodo 2	Nodo 3
UART0 (USB, consola)	GPIO1 / GPIO3	GPIO1 / GPIO3	GPIO1 / GPIO3
UART1 TX / RX	GPIO18 / GPIO19	GPIO18 / GPIO19	GPIO18 / GPIO19
UART2 TX / RX	GPIO17 / GPIO16	GPIO17 / GPIO16	GPIO17 / GPIO16
ADC señal contaminada x	GPIO34	—	—
ADC referencia s (opcional)	GPIO35	—	—
DAC salida de audio	—	GPIO25	GPIO25
LCD I2C SDA / SCL	—	—	GPIO21 / GPIO22
LED de error (pérdida o CRC)	GPIO4	GPIO4	GPIO4
LED RGB (método activo)	—	—	GPIO13 / GPIO14 / GPIO27
Botones	—	—	GPIO32 / GPIO33 (pull-up interno)

No usar: GPIO6–11 (flash), GPIO0, 2, 12 y 15 (arranque). No conectar nada a la UART0.

3. Enlaces entre nodos [Propuesta sobre Tarea 1]

Cada enlace es una UART en ambos sentidos, 921600 baudios, 8N1, lógica de 3.3 V. Las tres placas comparten tierra.

Enlace	Cables	Qué viaja
Nodo 1 ↔ Nodo 2	N1 GPIO18 → N2 GPIO19 · N2 GPIO18 → N1 GPIO19	N1→N2: datos x + DIAG · N2→N1: ACK de control
Nodo 1 ↔ Nodo 3	N1 GPIO17 → N3 GPIO16 · N3 GPIO17 → N1 GPIO16	N1→N3: datos x (y s si hay referencia) + DIAG · N3→N1: ACK de control
Nodo 2 ↔ Nodo 3	N2 GPIO17 → N3 GPIO19 · N3 GPIO18 → N2 GPIO16	N2→N3: salida FIR · N3→N2: ACK de control
Tierra	GND de las tres placas unidos	—

Uso de cada enlace a 921600 baudios [Tarea 1]:

Enlace	Bytes por bloque	Mínimo (baudios)	Uso
N1→N2: x + DIAG	2196	343 125	37.2 %
N1→N3: x, s + DIAG	4252	664 375	72.1 %
N2→N3: salida FIR	2094	327 188	35.5 %

115 200 baudios no alcanza.

4. Formato del paquete [Tarea 1]

```
[ cabecera 32 B | datos | CRC-32 4 B ]  →  codificación COBS  →  byte 0x00 delimitador
```

- **CRC-32/ISO-HDLC:** polinomio reflejado 0xEDB88320, valor inicial 0xFFFFFFFF, XOR final 0xFFFFFFFF. Se calcula sobre cabecera + datos, antes de COBS. Prueba: `"123456789"` → `0xCBF43926`.
- **COBS:** elimina los ceros internos para que 0x00 marque el fin de cada paquete.
- **Orden de bytes:** todos los campos van en little-endian (byte menos significativo primero).
- **Tamaño máximo tras COBS:** R + ⌊R/254⌋ + 2 bytes, donde R es cabecera + datos + CRC.

Cabecera (32 bytes)

Offset	Campo	Tipo	Valor o significado
0	identificador	uint16	0x4153 (“AS”) [Propuesta]
2	versión	uint8	1
3	tipo	uint8	ver tabla de tipos
4	origen	uint8	0 = PC, 1/2/3 = nodo
5	destino	uint8	0 = PC, 1/2/3 = nodo, 0xFF = todos
6	banderas	uint16	ver tabla de banderas
8	sesión	uint32	número aleatorio que el nodo 1 elige al arrancar
12	secuencia	uint32	+1 por cada paquete enviado en ese enlace
16	índice de primera muestra	uint64	posición de la primera muestra desde el inicio de la sesión
24	cantidad de muestras	uint16	512 en paquetes de datos, 0 en los demás
26	formato	uint8	0 = sin muestras, 1 = float32 [Propuesta]
27	canales	uint8	1 = solo x o solo FIR, 2 = x y s intercalados
28	longitud de datos	uint32	bytes de datos después de la cabecera

Tipos de mensaje [Propuesta]

Código	Nombre	Datos	Origen → destino
0x01	DATOS_X	512 × float32 = 2048 B	N1 → N2, N1 → N3
0x02	DATOS_XS	512 × 2 × float32 = 4096 B (x0, s0, x1, s1, …)	N1 → N3 con referencia
0x03	DATOS_FIR	512 × float32 = 2048 B	N2 → N3
0x10	DIAG	64 B	N1 → N2, N1 → N3
0x20	CONTROL	32 B	PC o N1 → cualquiera
0x21	ACK	32 B	respuesta a CONTROL
0x22	NACK	32 B	CONTROL rechazado

Banderas [Propuesta]

Bit	Significado
0	hay referencia limpia s en este bloque
1	reiniciar estados (filtros y estimador) a partir de este bloque
2	la configuración cambió a partir de este bloque
3	modo emisor: señal generada internamente, no capturada por ADC

Tamaños por paquete [Tarea 1]

Tipo	Datos (B)	Cabecera + datos + CRC (B)	En el cable (B)
DATOS_X o DATOS_FIR	2048	2084	2094
DATOS_XS	4096	4132	4150
DIAG	64	100	102
CONTROL, ACK, NACK	32	68	70

5. Paquete DIAG (64 bytes) [Propuesta sobre Tarea 1]

El nodo 1 envía un DIAG por cada bloque. Su campo “índice” en la cabecera coincide con el del bloque de datos que describe.

Offset	Campo	Tipo	Significado
0	fs	uint32	Hz
4	N	uint16	tamaño de FFT
6	hop	uint16	avance
8	tipo de ruido	uint8	ver sección 6
9	filtro sugerido nodo 2	uint8	ver sección 6
10	método sugerido nodo 3	uint8	ver sección 6
11	estado del estimador	uint8	0 = calentando (menos de 8 bloques), 1 = listo
12	frecuencia de interferencia	float32	Hz (0 si no aplica)
16	inicio de banda de ruido	float32	Hz (0 si no aplica)
20	fin de banda de ruido	float32	Hz (0 si no aplica)
24	piso de ruido	float32	dB
28	nivel DC	float32	estimado, en unidades normalizadas
32	energía del bloque	float32	Σx^2
36	SNR estimada	float32	dB, sin referencia
40	tiempo de proceso	uint32	µs que tardó el nodo 1 en este bloque
44	bloques perdidos	uint32	contador acumulado
48	errores de CRC	uint32	contador acumulado
52	desbordamientos de búfer	uint32	contador acumulado
56	saturaciones	uint32	contador acumulado (ADC fuera de rango)
60	id de configuración	uint32	+1 cada vez que cambia algún parámetro

6. Qué manda el nodo 1 según el ruido detectado [Propuesta]

Código	Tipo de ruido	Filtro nodo 2	Método nodo 3	Campos DIAG que se usan
0	ninguno o desconocido	0 = pasa todo	0 = pasa todo	—
1	tonal (interferencia sinusoidal)	2 = notch IIR	1 = picos	frecuencia de interferencia
2	60 Hz y armónicos	3 = peine 60 Hz	1 = picos	frecuencia de interferencia = 60
3	banda ancha (blanco)	1 = FIR pasa bajas	2 = resta espectral	piso de ruido
4	banda limitada	1 = FIR pasa bajas, o 2 = notch si la banda es angosta	2 = resta espectral con protección de tonos	inicio y fin de banda
5	impulsivo	1 = FIR pasa bajas	2 = resta espectral	energía del bloque

Códigos de filtro del nodo 2: 0 pasa todo · 1 FIR pasa bajas · 2 notch IIR (frecuencia desde DIAG) · 3 peine 60 Hz. **Códigos de método del nodo 3:** 0 pasa todo · 1 picos · 2 resta espectral · 3 top-k.

Los nodos 2 y 3 siguen la sugerencia del DIAG si están en modo automático. En modo manual usan lo que se configuró por CONTROL.

7. Paquetes CONTROL, ACK y NACK (32 bytes) [Propuesta]

Offset	Campo	Tipo	Significado
0	comando	uint8	1 = SET, 2 = GET, 3 = START, 4 = STOP, 5 = RESET_CONTADORES
1	parámetro	uint8	ver tabla de parámetros
2	reservado	uint16	0
4	valor	float32	valor nuevo (SET) o actual (respuesta a GET)
8	id de configuración	uint32	el que queda después del cambio
12	índice efectivo	uint64	primera muestra en la que rige el cambio (en ACK)
20	reservado	12 B	0

Parámetros configurables sin recompilar

Id	Parámetro	Nodo	Unidad
1	modo automático (1) o manual (0)	2, 3	—
2	filtro del nodo 2	2	código de la sección 6
3	método del nodo 3	3	código de la sección 6
4	frecuencia del notch	2	Hz
5	factor Q del notch	2	—
6	α de la resta espectral	3	—
7	ganancia mínima	3	0–1
8	umbral de detección de picos	1, 3	veces el piso
9	atenuación de picos	3	dB
20–22	frecuencias de los tonos 1, 2 y 3 (emisor)	1	Hz
23	frecuencia de la interferencia (emisor)	1	Hz
24	SNR objetivo (emisor)	1	dB

Cómo se cambia un parámetro

- **Consola USB de cualquier nodo:** comandos de texto, por ejemplo `set 4 2150`, `get 4`, `start`, `stop`, `status`. El nodo aplica el cambio y responde por la misma consola.
- **Desde el nodo 1:** reenvía el cambio como paquete CONTROL al nodo que corresponda y espera el ACK. Así el operador solo necesita hablar con el nodo 1 durante la defensa.
- **Botones del nodo 3:** un botón cambia el método, el otro alterna entre modo automático y manual.

Todo cambio queda en el id de configuración y en el índice efectivo, para poder relacionarlo con las métricas.

8. Qué hace el receptor con cada paquete [Tarea 1 + Propuesta]

1. Acumula bytes hasta recibir 0x00.
2. Si se pasó de 4150 bytes sin delimitador: descarta y suma un error de CRC.
3. Decodifica COBS. Si falla o el resultado mide menos de 36 bytes: descarta y suma un error de CRC.
4. Verifica el CRC. Si no coincide: descarta, suma un error de CRC y enciende el LED de error.
5. Verifica identificador, versión, destino (el propio número o 0xFF) y que la longitud declarada coincida.
6. **Sesión:** si cambió, reinicia estados, contadores de secuencia y estimador.
7. **Secuencia:** si salta, suma los bloques perdidos, enciende el LED de error un segundo, marca las métricas de ese tramo como no válidas y reinicia estados con una transición suave en la salida.
8. **Índice:** debe ser el anterior + 512. Si no, se trata igual que una pérdida.
9. **Búfer lleno:** descarta el bloque que llega, suma un desbordamiento. No se pide reenvío: el audio vencido no se recupera.

Los contadores (bloques perdidos, errores de CRC, desbordamientos, saturaciones) se muestran en la consola USB y en la LCD del nodo 3.

9. Métricas y referencia limpia [Tarea 1]

- Si el bloque trae referencia (bandera 0), el nodo 3 calcula MSE, SNR de entrada y salida, ΔSNR y energía útil conservada para las dos rutas.
- Para comparar con la salida FIR se compensa su retardo de 18 muestras.
- Sin referencia (por ejemplo en P6), las métricas que necesitan s se muestran como “N/D” y se usan las estimaciones sin referencia: SNR estimada por piso de ruido y atenuación en la banda detectada.

10. Código compartido [Propuesta]

- Un solo archivo de definiciones para el firmware (`protocolo.h` , usado por los tres nodos en C++) y su equivalente para los scripts de la PC (`protocolo.py`), con las mismas constantes, códigos y estructuras de este documento.
- Ningún nodo define valores propios del protocolo fuera de esos archivos.
- Las funciones de COBS y CRC de la Tarea 1 se reutilizan y se prueban con el valor de prueba del CRC antes de cada integración.